University Alumni Database Platform

Project Overview and Development Flow

Stack: Laravel | MySQL | HTML, CSS and JavaScript

Approach: API-first, mobile-first, privacy-first

Planning document | October 2026

1. What the Alumni Database Is About

The Alumni Database Platform is a secure web application that keeps a university connected to its graduates, and keeps graduates connected to one another. At its simplest it is a searchable directory of alumni. At its best it becomes a living community where graduates find classmates, mentor students, share jobs, attend events, and give back to the institution.

An earlier PHP version allowed alumni to connect, view each other's profiles, and search by name, matric number, department and year. That backup was lost, so this document defines a stronger rebuild from scratch, with better structure, security and features.

Problems the platform solves

- The university loses contact with graduates after they leave, so records go stale.
- Alumni cannot easily find former classmates or professionals in their field.
- Students and recent graduates have no structured way to reach experienced alumni for guidance.
- Reunions, fundraising and announcements depend on scattered WhatsApp groups and spreadsheets.
- The university lacks reliable data on graduate employment, needed for accreditation and reporting.

Who uses it

Role	What they do
Alumnus	Registers, builds a profile, searches and connects with others, messages, joins groups, posts and applies to jobs, attends events, donates.
Student / Recent graduate	Finds mentors, browses career paths and job listings (limited access).
Chapter / Class representative	Manages a group or set, posts announcements, organises local events.
Moderator	Reviews reports, removes inappropriate content, handles verification disputes.
Administrator	Imports graduate records, manages users and roles, views analytics, configures the system.

2. Goals and Guiding Principles

- **Verified community.** Only real graduates join, checked against the university's own records.
- **Privacy first.** Every alumnus controls which details are visible and to whom, in line with the Nigeria Data Protection Act 2023.
- **Mobile first.** Light pages, compressed images and an installable PWA, because most users are on phones with costly data.
- **Reasons to return.** Jobs, mentorship, events and news, so the directory is not visited once and forgotten.
- **Safe by design.** Hashed passwords, HTTPS, validation, rate limiting, role-based access and daily backups.
- **Built to grow.** Phased delivery, clean code and an API that can later serve a mobile app.

3. Technology Stack

Layer	Choice	Why
Backend framework	Laravel (PHP 8.2+)	Built-in authentication, validation, migrations, queues, mail, scheduling and testing. Familiar to PHP developers.
Database	MySQL 8	Reliable, widely hosted, supports FULLTEXT search, JSON columns and strong indexing.
Frontend	HTML, CSS, JavaScript (ES modules)	Lightweight pages calling the API with fetch(). A CSS framework such as Tailwind speeds up consistent design.
API auth	Laravel Sanctum	Simple token or cookie authentication for the JavaScript frontend.
Queues and cache	Database queue first, Redis later	Sends emails and bulk imports in the background without slowing users down.
File storage	Local disk, then S3-compatible	Profile photos, event images and documents.
Payments	Paystack or Flutterwave	Local cards, bank transfer and USSD for donations and event tickets.
Version control	Git and GitHub/GitLab	Full history of the code, so a lost backup never happens again.

Why API-first?

The Laravel backend returns only JSON. The HTML, CSS and JavaScript frontend requests data from it with `fetch()`. This keeps logic and design separate, lets the frontend be improved without touching business rules, and allows a mobile app to reuse the same API later.

4. System Architecture

Requests flow from the user's browser to Laravel, which validates them, applies permissions and talks to MySQL. Background jobs handle slow tasks.

1 Browser / PWA
HTML, CSS, JS pages. Service worker for offline shell.

2 HTTPS and Web Server
Nginx or Apache, rate limiting, security headers.

3 Laravel API
Routes, controllers, form requests, policies, resources.

4 MySQL Database
Users, profiles, records, groups, jobs, events, logs.

5 Queue Workers
Emails, SMS, CSV imports, notifications.

6 Storage and Backups
Images, files, nightly database dumps stored off-server.

Folder structure (Laravel)

```
app/
  Http/Controllers/Api/     AuthController, ProfileController, SearchController ...
  Http/Requests/            Validation rules for each form
  Http/Resources/           Shapes the JSON returned (respects privacy settings)
  Models/                   User, Profile, GraduateRecord, Group, Job, Event ...
  Policies/                 Who may view or edit what
  Jobs/ and Mail/           Background tasks and email templates
database/
  migrations/               Table definitions (version-controlled)
  seeders/                  Departments, roles, test data
routes/api.php              All API endpoints
public/ (frontend)
  index.html  about.html  features.html  how-it-works.html
  events.html  news.html  contact.html  faq.html  privacy.html  terms.html
  login.html  register.html  forgot-password.html
  app/  dashboard.html  directory.html  profile.html  messages.html ...
  css/  js/modules/  sw.js  manifest.json  sitemap.xml
```

5. Features by Phase

Delivering in phases gets a solid, usable system live early and lets real feedback guide later work.

Phase	Features
1. Foundation (MVP)	Public website (landing, about, features, FAQ, contact) leading to login and registration; verified registration against graduate records; email and phone confirmation; login and password reset; rich profiles; per-field privacy; search and filters (name, matric number, department, year, industry, location, skills); admin dashboard with CSV import; automated backups.
2. Community	Connection requests; direct messaging; in-app and email notifications; chapters and groups; "Find my set" for classmates; "People you may know"; report and block; achievement feed.
3. Career and Events	Job and opportunity board with referrals; mentorship matching; Ask an Alum Q&A; events with RSVP, tickets and QR check-in; class reunion countdowns.
4. Impact and Polish	Alumni map; digital alumni ID with QR verification; donations and campaigns; business directory; digital yearbook; tracer surveys; university analytics and reports; PWA polish; optional AI helpers.

6. Standout Features

These are the features that make the platform memorable and useful beyond a basic directory.

Feature	Value
Verified digital alumni ID with QR code	Anyone can scan to confirm a person is a genuine graduate. Useful at events, for discounts and campus access.
Mentorship matching	Alumni opt in as mentors by expertise; students request them. A strong selling point to the university and funders.
Career path explorer	Charts showing where graduates of each department end up: industries, employers, cities.
Alumni map	Interactive map of where alumni live, clustered by city, linked to local chapters.
Find my set	See classmates by department and year, and invite those who have not joined.
WhatsApp-friendly sharing	Clean preview links for profiles, events and jobs, since WhatsApp is where most alumni already talk.
Graduate tracer surveys	Built-in surveys that produce the employment data universities need for accreditation.
University analytics	Employment rates, top employers, geographic spread and giving trends, exportable as reports.
Gamification	Profile completion bar and badges (Mentor, Top Contributor, Class Rep) to encourage participation.

7. Public Website and Landing Pages

The platform is not just a login screen. Anyone who visits the website, whether an alumnus, a student, a parent, a potential donor or a university partner, first lands on a **public website** made of several pages that explain what the alumni database is, what it offers and why they should join. From there, the visitor chooses **Login** or **Register**, and the matching page opens.

Visitor journey

1 Visitor arrives
Opens the website link, shared on WhatsApp, social media or the university site.

2 Lands on Home page
Sees the introduction, key features, live statistics and clear buttons.

3 Explores public pages
About, Features, How It Works, Events, News, FAQ and Contact via the menu.

4 Clicks a button
Login or Register, available in the menu, the top banner and the bottom of each page.

5 Goes to the right page
Login button opens the login page. Register button opens the registration page.

6 Enters the platform
After login, taken to the dashboard (alumni) or admin panel (staff), by role.

Public pages

Page	Purpose and content
Home (index)	First impression. Headline, short explanation of the alumni database, Login and Register buttons, live statistics, feature highlights, how it works, alumni stories, upcoming events and a final call to action.
About	The purpose of the platform, the university's vision for its alumni, the team or alumni office behind it, and the benefits for graduates and the university.
Features	A clear tour of what members can do: directory and search, profiles, messaging, groups, jobs, mentorship, events, digital ID, giving. Uses icons and screenshots.
How It Works	Three or four simple steps: verify with your matric number, build your profile, connect, grow. Explains the verification process and privacy controls.
News and Events	Public announcements, reunions and homecoming dates, with a prompt to register for full details and RSVP.
Alumni Stories	Featured graduates and success stories, shown only with the person's consent.
Chapters	List of alumni chapters by city or country and how to join them.
Giving / Support	Explains scholarship and project campaigns and links to donate.
FAQ	Answers on eligibility, verification, privacy, lost matric number, account problems and costs.
Contact	Contact form, alumni office email and phone, location and social media links.
Privacy Policy and Terms	How data is collected, used and protected (NDPA 2023), and the rules of use.
Verify ID	Public result page opened by scanning an alumni ID QR code; confirms authenticity without exposing private data.
Error pages	Friendly 404 and maintenance pages with links back home.

Anatomy of the Home page

Section	What it shows
Navigation bar	Logo, page links, and two prominent buttons: Login and Register. Becomes a mobile menu on phones.
Hero banner	Strong headline, one-sentence explanation, Register (primary) and Login (secondary) buttons, and a background photo of the campus or graduates.
Live statistics	Total alumni, departments, graduation years and countries, loaded from a public statistics endpoint (totals only, never personal data).
Features overview	Six to eight feature cards with icons: search, connect, mentorship, jobs, events, digital ID.
How it works	Three steps from verification to joining the community.
Stories and testimonials	Short quotes and photos from consenting alumni.
Upcoming events and news	Next two or three public events and latest announcements.
Final call to action	Repeat of the Register and Login buttons so the visitor does not have to scroll back up.
Footer	Quick links, contact details, social media, privacy and terms links, university name and logo.

How the Login and Register buttons work

Every public page shares the same navigation bar and footer through a small JavaScript module, so the buttons are identical everywhere. The Login button opens `login.html` and the Register button opens `register.html`. If a visitor is already signed in, both buttons are replaced by a single Dashboard button so they are not asked to log in again. After a successful login, the system redirects by role.

```
// js/modules/nav.js  (loaded on every public page)
const token = localStorage.getItem('token');
const login = document.querySelector('#btn-login');
const register = document.querySelector('#btn-register');

if (token) {                        // already signed in
  login.textContent = 'Dashboard';
  login.href = '/app/dashboard.html';
  register.hidden = true;
} else {
  login.href = '/login.html';
  register.href = '/register.html';
}
```

Rules for the public pages

- **No personal data.** Public pages never show individual alumni details; only totals, and stories that the person has agreed to share.
- **Fast and light.** Plain HTML and CSS with compressed images, so they load quickly on weak mobile networks.
- **Search-engine and share friendly.** Page titles, meta descriptions, Open Graph tags (for WhatsApp previews) and a sitemap so people can find the site.
- **Admin-managed content.** News, events, stories, FAQs and statistics text are edited from the admin panel, so the website never needs code changes to stay current.
- **Consistent design.** One shared style, colours, logo and button design across public pages, login, registration and the member area.
- **Accessible and bilingual-ready.** Readable fonts, good contrast and structure that can support more than one language later.
- **Consent and cookies.** A short notice and links to the privacy policy on first visit.

Public API endpoints

These endpoints need no login and are rate limited. They feed the landing pages.

```
GET   /api/public/stats          totals: alumni, departments, countries, chapters
GET   /api/public/events         upcoming public events
GET   /api/public/news           latest announcements
GET   /api/public/stories        consented alumni spotlights
GET   /api/public/faqs           frequently asked questions
POST  /api/public/contact        contact form message (spam protected)
GET   /api/verify-id/{code}      result of scanning an alumni ID QR code
```

8. Database Design (MySQL)

The most important design decision is keeping **graduate_records** (the university's trusted master list) separate from **users** (people who actually signed up). Registration succeeds only when the details match a graduate record.

Table	Purpose and key columns
graduate_records	Trusted master list. matric_number (unique), full_name, department_id, faculty, graduation_year, degree, claimed_by_user_id.
users	Login account. email (unique), phone, password_hash, role, status, graduate_record_id, email_verified_at.
profiles	Public details. user_id, photo, headline, bio, current_job, employer, industry, city, country, social links.
privacy_settings	Per-field visibility: user_id, field_name, visibility (public, alumni, connections, private).
departments / faculties	Lookup tables used for filters and records.
education / work_experience	Extra history entries linked to a user.
skills / user_skills	Skill tags and the many-to-many link to users.
connections	requester_id, receiver_id, status (pending, accepted, blocked).
messages / conversations	Direct messages with read status.
groups / group_members	Chapters, sets, faculties and interest groups.
posts / comments / reactions	Achievement feed and announcements.
jobs / job_applications	Opportunity board and referrals.
mentorships	mentor_id, mentee_id, topic, status.
events / rsvps / tickets	Events, attendance and QR check-in.
donations / campaigns	Fundraising, payment reference, amount, status.
notifications	In-app and email notification records.
news_articles / testimonials / faqs	Content shown on the public website, managed from the admin panel.
contact_messages	Messages sent from the public contact form, with status and reply tracking.
audit_logs	Who did what and when, for security and accountability.

Indexing for fast search

Search is the heart of the system, so index the columns people filter on and add a FULLTEXT index for names.

```
-- Example migration (simplified)
Schema::create('graduate_records', function (Blueprint $t) {
    $t->id();
    $t->string('matric_number')->unique();
    $t->string('full_name');
    $t->foreignId('department_id')->constrained();
    $t->year('graduation_year')->index();
    $t->unsignedBigInteger('claimed_by_user_id')->nullable();
    $t->timestamps();
    $t->fullText('full_name');
});
```

9. The Development Flow, Step by Step

This is the order of work from an empty folder to a live system.

1 Plan and Design
Confirm requirements with the university, sketch screens, finalise the data model.

2 Set Up Environment
PHP 8.2, Composer, MySQL, Git repo, .env files for local and live.

3 Build the Database
Write migrations and seeders for all Phase 1 tables.

4 Authentication
Registration, login, email verification, password reset with Sanctum.

5 Verification Engine
Match matric number, name and year against graduate_records.

6 Profiles and Privacy
Profile CRUD, photo upload, per-field visibility.

7 Search and Directory
Filters, FULLTEXT search, pagination, privacy-aware results.

8 Admin Panel
CSV import, user approval, roles, reports, statistics.

9 Public Website
Landing, about, features, FAQ and contact pages; Login and Register buttons.

10 App Pages and PWA
Login, register and member pages consuming the API; PWA setup.

11 Test and Deploy
Automated tests, security checks, live server, HTTPS, backups.

12 Launch and Iterate
Onboard a pilot group, gather feedback, start Phase 2.

Getting started commands

```
composer create-project laravel/laravel alumni-platform
cd alumni-platform
php artisan install:api                 # sets up Sanctum for API auth
# configure .env  ->  DB_CONNECTION=mysql, DB_DATABASE=alumni_db ...
php artisan make:model GraduateRecord -mf   # model + migration + factory
php artisan make:controller Api/AuthController
php artisan make:request RegisterRequest
php artisan make:policy ProfilePolicy
php artisan migrate --seed
php artisan test
```

10. Registration and Verification Flow

This flow protects the community from fake accounts and keeps the university's data trustworthy.

Step	What happens
1	Admin imports graduate records from the university's spreadsheet (CSV) into graduate_records.
2	Visitor clicks Register on any public page, lands on the registration page, and enters matric number, full name, department and graduation year.
3	System looks for a matching, unclaimed graduate record. Fuzzy matching tolerates small spelling differences.
4	On a match, the visitor sets email, phone and password. On no match, they may submit a manual verification request with supporting details.
5	A confirmation link is emailed (and an OTP sent by SMS if enabled). The account activates once confirmed.
6	The record is marked as claimed so no one else can register with the same matric number.
7	The new alumnus is guided through completing a profile and setting privacy choices.
8	Manual requests go to a moderator queue for approval or rejection.

Example API endpoints

```
POST   /api/register              verify against graduate_records, create user
POST   /api/login                 returns token
GET    /api/profile/{id}          profile (fields filtered by privacy settings)
PUT    /api/profile               update own profile
GET    /api/alumni?name=&dept=&year=&industry=&city=&skills=&page=
POST   /api/connections           send request
POST   /api/messages              send message
GET    /api/admin/stats           admin only
POST   /api/admin/import          CSV upload of graduate records
```

11. Search Design

Search must be fast, forgiving and respectful of privacy.

- **Filters:** name, matric number, department, graduation year, industry, city or country, employer and skills, combinable.
- **Name search:** MySQL FULLTEXT for relevance, with a LIKE fallback for partial names; optional SOUNDEX or trigram matching for misspellings.
- **Privacy-aware:** results hide any field the person has restricted; users who opted out of the directory do not appear.
- **Pagination and caching:** return 20 results per page; cache common filter lists such as departments and years.
- **Abuse protection:** rate limit searches and log unusual bulk scraping.
- **Later upgrade:** move to Meilisearch if the user base grows large.

12. Security and Privacy

Area	Measures
Accounts	Bcrypt or Argon2 password hashing, email verification, login throttling, optional two-factor authentication for admins.
Data protection	HTTPS everywhere, input validation with Form Requests, Eloquent (prevents SQL injection), CSRF and XSS protection, secure file upload checks.
Access control	Roles and Laravel Policies so each action is checked on the server, never just hidden in the interface.
Privacy and compliance	Consent at sign-up, per-field visibility, data export, account deletion, retention policy, aligned with NDPA 2023.
Accountability	Audit logs for admin actions, report and block tools, moderation queue.
Resilience	Automated daily MySQL backups kept off-server, weekly restore tests, Git for all code, monitoring and error alerts.

13. Frontend Plan (HTML, CSS, JavaScript)

- **Public pages:** home, about, features, how it works, news and events, FAQ, contact, privacy and terms, each with the shared Login and Register buttons.
- **Access pages:** login, register, forgot password.
- **Member pages:** dashboard, directory with filters, profile view and edit, connections, messages, groups, jobs, events, admin screens.
- **Structure:** reusable JS modules (api.js, auth.js, search.js, ui.js) that wrap fetch() calls and handle tokens and errors.
- **Design:** mobile-first, accessible, high contrast, light and dark themes, Tailwind or a small CSS framework.
- **PWA:** manifest.json and a service worker so the site can be installed and loads quickly on weak networks.
- **Performance:** lazy-load images, compress photos on upload, paginate lists, minify assets.
- **Sharing:** Open Graph tags so shared links preview nicely on WhatsApp and social media.

Example: calling the search API

```
export async function searchAlumni(filters, page = 1) {
  const params = new URLSearchParams({ ...filters, page });
  const res = await fetch(`/api/alumni?${params}`, {
    headers: { Accept: 'application/json',
               Authorization: `Bearer ${localStorage.getItem('token')}` }
  });
  if (!res.ok) throw new Error('Search failed');
  return res.json();      // { data: [...], meta: { current_page, last_page } }
}
```

14. Testing and Quality

- **Automated tests:** Laravel feature tests for registration, verification, privacy rules and search; run on every change.
- **Security review:** test access control (can a user read another user's hidden fields?), file uploads and rate limits.
- **Performance test:** seed tens of thousands of fake alumni and confirm search stays fast.
- **User trial:** pilot with one department or graduating set and collect feedback before opening to everyone.

15. Deployment and Operations

- **Hosting:** a VPS or university server running Nginx, PHP-FPM and MySQL 8; managed hosting is an alternative.
- **Environments:** local, staging and production, each with its own .env and database.
- **Release process:** push to Git, run tests, deploy, run `php artisan migrate`, clear and rebuild caches.
- **Scheduled tasks:** Laravel scheduler for nightly backups, reminder emails, event notices and cleanup.
- **Monitoring:** error tracking, uptime checks, and disk and database alerts.

16. Suggested Timeline

Period	Milestone
Weeks 1-2	Requirements, data model, wireframes, environment and Git setup; gather text, photos and logo for the public pages.
Weeks 3-6	Phase 1: database, authentication, verification, profiles, privacy, search, admin panel, and the public website pages.
Weeks 7-8	Frontend polish, PWA, testing, backups, deployment and pilot launch.
Weeks 9-12	Phase 2: connections, messaging, notifications, groups.
Weeks 13-18	Phase 3: jobs, mentorship, events and QR check-in.
Weeks 19-26	Phase 4: map, digital ID, donations, analytics, optional AI features.

Timings assume one developer working steadily and will vary with team size and university approvals.

17. Risks and How to Handle Them

Risk	Mitigation
Fake or duplicate accounts	Verification against graduate records, claimed-record flag, moderation queue.
Poor or incomplete university data	Data-cleaning step on import, manual verification route, admin editing tools.
Low adoption	Launch with a pilot set, class reps as champions, useful features (jobs, mentorship) early, WhatsApp sharing.
Privacy complaints	Private-by-sensible-default settings, clear consent, easy deletion and export.
Data loss	Automated off-server backups, tested restores, Git for code.
Scope creep	Strict phases; build only after Phase 1 is stable and used.

18. Immediate Next Steps

- Confirm with the university which graduate data is available (matric number, name, department, year) and in what format.
- Decide hosting (university server or commercial VPS) and domain name.
- Set up the Laravel project, MySQL database and Git repository.
- Build the migrations and the registration-with-verification flow first, since everything depends on it.
- Collect content for the public website: university description, logo, photos, features text, FAQs and contact details.
- Design and build the public landing pages, then the login and registration pages they lead to.
- Design the directory and profile screens, then build the search.